WELT 3 · DOPPELSTUNDE 18 · IDE-LEVEL 3.1 · DEKO-WERKSTATT

W3·1 – Das Labyrinth: Rekursion

Dateien: `AQ_W3_L3_Labyrinth_Geruest.txt` (Klasse Rekursion, Level 3.1) · `AQ_W3_Fraktale_Geruest.txt` (Deko-Werkstatt) · Im Lab: **Level 3.1** und **Level 3.2**
Niveau (KC 2017): nur Leistungskurs (eA) – im Grundkurs nicht abiturrelevant

Unter der Rangliste öffnet sich eine Wendeltreppe ins **Labyrinth**: Gänge, die sich verzweigen, Kammern hinter Kammern, Nebel, Sackgassen. Wer hier einen Plan für jeden einzelnen Gang schreiben wollte, würde nie fertig. Die Zunft der Kartografen hat deshalb einen eigenen Zauber: Sie beschreibt einen großen Gang durch einen **kleineren Gang derselben Art** – bis der Gang so klein ist, dass die Antwort sofort feststeht.

Diesen Zauber nennt man **Rekursion**: Eine Methode ruft **sich selbst** auf. Heute lernst du, rekursive Methoden zu lesen, Schritt für Schritt zu verfolgen (wie im Abitur in einer **Rekursionstabelle mit Rekursionstiefe**), die Anzahl der Aufrufe zu bestimmen und selbst rekursiv zu programmieren. In fast jeder Abituraufgabe der letzten Jahre kam Rekursion vor.

Teil A – Bilder der Rekursion

Aufgabe 0: Matroschka, Spiegel und Vorfahren

PAPIER ★



Abbildung 1: Drei Bilder für dieselbe Idee

a) **Matroschka:** Du willst wissen, wie viele Puppen ineinanderstecken. Beschreibe ein Verfahren, das nur „eine Puppe öffnen“ und „das Verfahren für die kleinere Puppe wiederholen“ benutzt. Woran erkennst du, dass du aufhören kannst? Was wäre, wenn es keine kleinste, massive Puppe gäbe?

b) **Spiegel im Spiegel:** Zwei Spiegel stehen sich gegenüber. Die Idee „Bild im Bild im Bild …“ ist unendlich. Begründe, warum das echte Bild trotzdem irgendwann aufhört.

c) **Vorfahren:** Erkläre, was ein „Vorfahre“ einer Person X ist, **ohne** das Wort selbst zu benutzen – aber so, dass auch Ur-Ur-Großeltern erfasst werden. Tipp: Du brauchst das Wort „Elternteil“ zweimal.

Infoteil 1: Basisfall und Rekursionsschritt

Eine Methode heißt **rekursiv**, wenn sie sich selbst aufruft. Damit das nicht endlos läuft, braucht sie **zwei Bestandteile**:

Bestandteil	Bedeutung	in quadratsumme
Basisfall (Rekursionsanfang)	ein so kleiner Fall, dass die Antwort sofort feststeht – **kein** Selbstaufruf	n = 0: Ergebnis 0
Rekursionsschritt	das Problem wird auf ein **kleineres Problem derselben Art** zurückgeführt und per Selbstaufruf gelöst	$1^2 + \ldots + n^2 = n \cdot n + (1^2 + \ldots + (n-1)^2)$

```
// 1*1 + 2*2 + ... + n*n  (n >= 0)
public static int quadratsumme(int n) {
    if (n == 0) {
        return 0;                       // Basisfall
    }
    int rest = quadratsumme(n - 1);     // Rekursionsschritt
    return n * n + rest;
}
```

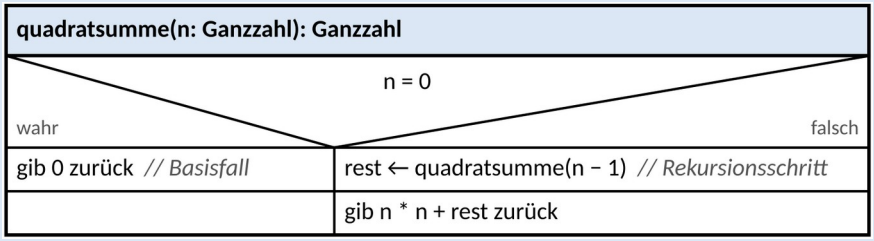


Abbildung: Struktogramm von quadratsumme nach EH § 5

- **Im Struktogramm** (EH § 5) ist ein Selbstaufruf eine ganz normale Anweisung: `rest ← quadratsumme(n − 1)` oder direkt in einem Ausdruck, z. B. „gib n * n + quadratsumme(n − 1) zurück“. Ein besonderes Symbol gibt es nicht.
- **Merke:** Jede Rekursion braucht einen Basisfall, und jeder Selbstaufruf muss dem Basisfall **näherkommen** (hier: n wird um 1 kleiner). Den Basisfall prüfst du **vor** dem Selbstaufruf.

Infoteil 2: Der Aufrufstapel

Jeder Aufruf bekommt einen **eigenen Rahmen** mit eigenen Parametern und lokalen Variablen – drei gleichzeitig lebende Variablen n stören sich nicht. Der Aufrufer **wartet**, bis der aufgerufene Rahmen sein Ergebnis liefert. Die Rahmen liegen auf einem **Stapel** (engl. stack): Der zuletzt begonnene Aufruf wird als erster fertig.

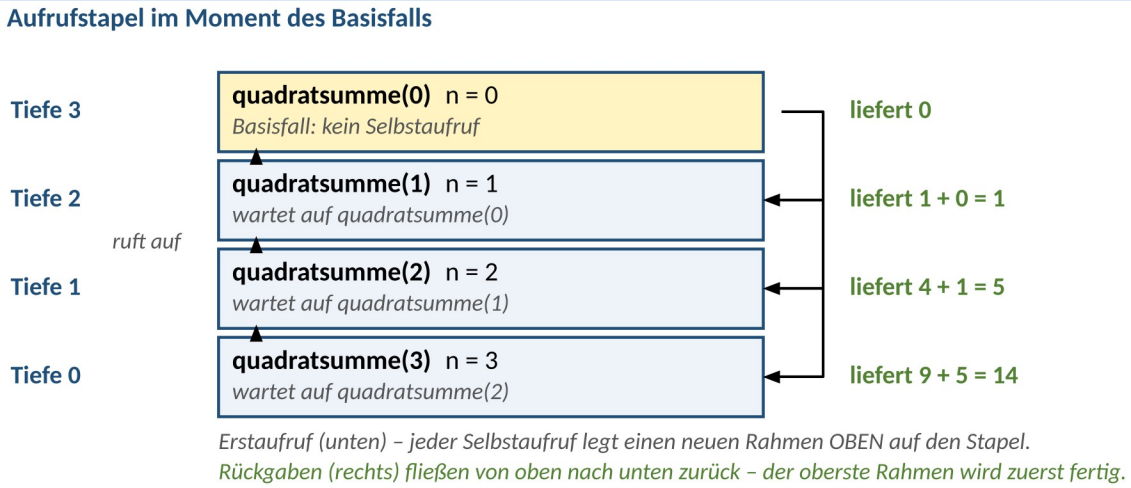


Abbildung 2: Aufrufstapel beim Aufruf quadratsumme(3)

Der Stapel ist begrenzt. Fehlt der Basisfall oder wird er nie erreicht, kommen immer neue Rahmen dazu, bis der Speicher voll ist: Java bricht mit **StackOverflowError** ab (Stapelüberlauf) – der rekursive Zwilling der Endlosschleife.

Infoteil 3: Die Rekursionstabelle – so verlangt es das Abitur

Im Abitur dokumentierst du einen rekursiven Ablauf in einer Tabelle: **eine Zeile pro Aufruf**, in der Reihenfolge, in der die Aufrufe **beginnen**. Die Spalte **Rekursionstiefe** zählt, wie tief ein Aufruf im Stapel liegt: Der **Erstaufruf hat die Tiefe 0**, jeder Selbstaufruf erhöht die Tiefe um 1.

Rekursionstiefe *)	Aufruf	n	Rückgabe
0	quadratsumme(3)	3	9 + 5 = 14
1	quadratsumme(2)	2	4 + 1 = 5
2	quadratsumme(1)	1	1 + 0 = 1
3	quadratsumme(0)	0	0 (Basisfall)

**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

- **Abstieg:** Die Spalten Tiefe, Aufruf und Parameter füllst du von oben nach unten – bis zum Basisfall.
- **Aufstieg:** Die Spalte Rückgabe füllst du danach **von unten nach oben**: Erst wenn quadratsumme(0) die 0 liefert, kann quadratsumme(1) rechnen: 1 + 0 = 1.
- Anzahl der Aufrufe hier: 4 (Erstaufruf mitgezählt), größte Rekursionstiefe: 3.

Aufgabe 1: Das Zettel-Spiel im Spiegelsaal

PAPIER ★

Spielt die Methode `spiegel` im Team mit Zetteln nach: Wer einen Zettel `spiegel(k)` bekommt, notiert k, schreibt einen neuen Zettel `spiegel(k − 1)`, gibt ihn weiter und **wartet** (steht auf). Im Basisfall wird sofort geantwortet. Jede antwortende Person setzt sich wieder.

```
1  public static String spiegel(int n) {
2      if (n == 0) {
3          return "|";
4      }
5      return n + spiegel(n - 1) + n;
6  }
```

a) Dokumentiere den Aufruf `spiegel(3)` in einer Rekursionstabelle und gib den Rückgabewert an.

Rekursionstiefe *)	Aufruf	n	Rückgabe
0			

**) Man spricht bei einem rekursiven Aufruf davon, dass sich die Rekursionstiefe um 1 erhöht. Der Erstaufruf hat die Rekursionstiefe 0.*

b) Wie viele Personen braucht ihr für `spiegel(3)`, wie viele für `spiegel(n)`? Wie groß ist die größte Rekursionstiefe?

c) Ein Teammitglied ändert den Basisfall in Zeile 2 zu `n == 1` mit der Rückgabe `"1|1"`. Beurteile die Änderung: Was liefert `spiegel(3)`, und was passiert bei `spiegel(0)`?

Aufgabe 2: Rekursionstabellen im Gewölbe

PAPIER ★★

Zwei Methoden aus dem Zauberbuch der Kartografen (es gilt a ≥ 0, b > 0 und n ≥ 0):

```
public static int rest(int a, int b) {
    if (a < b) {
        return a;
    }
    return rest(a - b, b);
}
```

```
public static int geheim(int n) {
    if (n == 0) {
        return 0;
    }
    return geheim(n - 1) + 2 * n - 1;
}
```

a) Dokumentiere `rest(23, 5)` in einer Rekursionstabelle.

Rekursionstiefe *)	Aufruf	a	Rückgabe
0			

b) Beschreibe, was `rest(a, b)` berechnet. Gib an, wie viele Aufrufe `rest(100, 7)` insgesamt auslöst. Was fällt dir an der Spalte Rückgabe auf?

c) Dokumentiere `geheim(4)` in einer Rekursionstabelle (Spalten wie in a), Spalte a heißt hier n).

Rekursionstiefe *)	Aufruf	n	Rückgabe
0			

d) Stelle eine Vermutung auf, was `geheim(n)` allgemein liefert, und begründe sie mithilfe des Rekursionsschritts.

Aufgabe 3: Hin- und Rückweg mit Fackeln

PAPIER ★★

Ein Held steigt mit Fackeln in den Schacht hinab und wieder hinauf. Die Methode **liefert nichts** – sie gibt nur aus.

```
public static void fackeln(int n) {
    if (n > 0) {
        println("an " + n);
        fackeln(n - 1);
        println("aus " + n);
    }
}
```

```
public static void fackeln2(int n) {
    if (n > 0) {
        fackeln2(n - 1);
        println("an " + n);
    }
}
```

a) Gib die vollständige Ausgabe von `fackeln(3)` an.

b) Gib die Ausgabe von `fackeln2(3)` an. Erläutere mit dem Aufrufstapel, warum die Zahlen hier aufsteigend erscheinen, obwohl mit 3 begonnen wird.

c) Wo steckt in `fackeln` der Basisfall? Begründe, dass `fackeln(n)` für jedes n ≥ 0 endet.

Infoteil 4: Rekursion oder Schleife?

Jede Wiederholung lässt sich mit einer Schleife **oder** rekursiv schreiben. Bei Reihungen und Zeichenketten übernimmt ein zusätzlicher Parameter i (die aktuelle Position) die Rolle der Zählvariablen. Damit die vorgegebene Signatur erhalten bleibt, ruft eine **Hüllmethode** die rekursive Hilfsmethode mit i = 0 auf:

```
// mit Schleife
public static boolean enthaelt(int[] r, int x) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] == x) {
            return true;
        }
    }
    return false;
}
```

```
// rekursiv mit Hilfsmethode
public static boolean enthaelt(int[] r, int x) {
    return enthaeltAb(r, x, 0);      // Hülle
}
public static boolean enthaeltAb(int[] r, int x, int i) {
    if (i == r.length) {
        return false;      // Basisfall: nichts mehr da
    }
    if (r[i] == x) {
        return true;       // Basisfall: gefunden
    }
    return enthaeltAb(r, x, i + 1);
}
```

	Schleife (iterativ)	Rekursion
Wiederholung durch	Schleifenkopf, Zählvariable	Selbstaufruf mit kleinerem Problem
Ende durch	Schleifenbedingung	Basisfall
Speicher	eine Variable i	ein Rahmen pro Aufruf auf dem Stapel
typischer Fehler	Endlosschleife	fehlender Basisfall → StackOverflowError
besonders geeignet	einfaches Durchlaufen	Probleme, die in **gleichartige Teilprobleme** zerfallen: Labyrinthe, Bäume, Teile und herrsche

Aufgabe 4: Bug-Hunt: Wenn der Stapel überläuft

PAPIER ★★

Jede Methode enthält **genau einen** Fehler. Es gilt jeweils n ≥ 1 bzw. s ≠ null.

```
// A: soll 1 * 2 * ... * n liefern
public static int produkt(int n) {
    int t = n * produkt(n - 1);
    if (n == 1) {
        return 1;
    }
    return t;
}
// B: soll die Länge von s liefern (ohne length() der ganzen Zeichenkette)
public static int laenge(String s) {
    if (s.equals("")) {
        return 1;
    }
    return 1 + laenge(s.substring(1));
}
// C: soll s rückwärts liefern, z. B. "Gold" -> "dloG"
public static String umdrehen(String s) {
    if (s.length() == 1) {
        return s;
    }
    return umdrehen(s.substring(1)) + s.charAt(0);
}
// D: soll zählen, wie oft 'x' in s vorkommt
public static int anzahlX(String s) {
    if (s.length() == 0) {
        return 0;
    }
    anzahlX(s.substring(1));
    if (s.charAt(0) == 'x') {
        return 1;
    }
    return 0;
}
```

a) Gib für jeden Aufruf das Ergebnis an – oder den Fehler, mit dem das Programm abbricht. **[4 BE]**

	Aufruf	Ergebnis bzw. Abbruch	richtig wäre
A	`produkt(3)`		6
B	`laenge("abc")`		3
C	`umdrehen("Gold")`		"dloG"
C	`umdrehen("")`		""
D	`anzahlX("xaxx")`		3

b) Beschreibe jeden Fehler in einem Satz und gib die Korrektur an. **[4 BE]**

c) Erläutere am Aufrufstapel, wie es bei A zum StackOverflowError kommt. Warum hilft es nicht, dass der Basisfall „eigentlich da“ ist? **[2 BE]**

Aufgabe 5: Schleifen-Bann: Aus Schleife wird Rekursion

PAPIER → IDE ★★

Im Labyrinth wirkt ein Bann: Schleifen funktionieren nicht. Schreibe beide Methoden **rekursiv** (ohne Schleife) – erst auf Papier, dann tippst du sie im Hauptprogramm-Bereich des Gerüsts ab und testest.

```
public static String sterne(int n) {
    String s = "";
    for (int i = 0; i < n; i++) {
        s = s + "*";
    }
    return s;
}
```

```
public static boolean alleGerade(int[] r) {
    for (int i = 0; i < r.length; i++) {
        if (r[i] % 2 != 0) {
            return false;
        }
    }
    return true;
}
```

a) Implementiere `sterne(n: Ganzzahl): Zeichenkette` rekursiv (n ≥ 0; `sterne(0)` liefert ""). **[2 BE]**

b) Implementiere `alleGerade` als Hüllmethode, die die rekursive Hilfsmethode `alleGeradeAb(r: Reihung von Ganzzahl, i: Ganzzahl): Wahrheitswert` aufruft, und `alleGeradeAb` selbst. **[3 BE]**

c) Stelle `alleGeradeAb` als Struktogramm nach EH § 5 dar. **[2 BE]**

d) Gib an, wie oft `alleGeradeAb` für r = {4, 8, 2} und für r = {4, 7, 2, 6} jeweils insgesamt aufgerufen wird. Prüfe danach in der IDE. **[1 BE]**

Infoteil 5: Baumrekursion – wenn sich eine Methode zweimal aufruft

Die Fibonacci-Zahlen 0, 1, 1, 2, 3, 5, 8, … sind definiert durch fib(0) = 0, fib(1) = 1 und fib(n) = fib(n − 1) + fib(n − 2). Hier ruft sich die Methode im Rekursionsschritt **zweimal** auf. Die Aufrufe bilden keinen Stapel in einer Linie mehr, sondern einen **Aufrufbaum**:

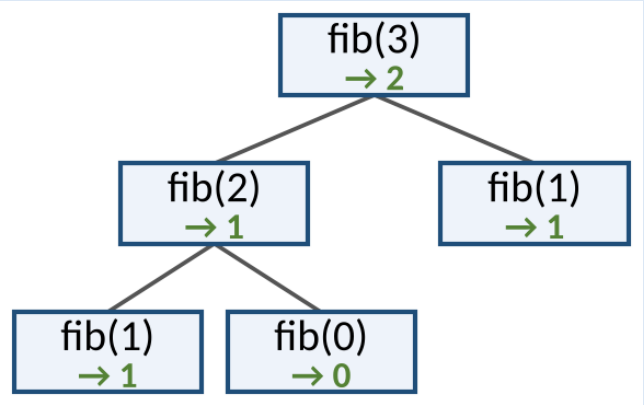


Abbildung 3: Aufrufbaum von fib(3) mit Rückgabewerten (grün)

- Jeder Kasten ist **ein Aufruf**. Die Anzahl aller Aufrufe ist die Anzahl der Kästen (hier 5, der Erstaufruf zählt mit). Die Rekursionstiefe eines Aufrufs ist seine **Ebene** im Baum (Wurzel = Tiefe 0).
- Abgearbeitet wird der Baum **von links nach rechts in die Tiefe**: fib(2) wird vollständig berechnet, bevor fib(1) rechts beginnt. Zu jedem Zeitpunkt liegt nur **ein Ast** des Baumes auf dem Stapel.
- **Zählen ohne Zeichnen:** Ein Aufruf verursacht sich selbst plus alle Aufrufe seiner Teilbäume. Wer die Aufrufzahlen kleinerer Fälle kennt, kann daraus die der größeren berechnen.
- **Gefahr:** Gleiche Teilprobleme werden **mehrfach** berechnet (fib(1) schon hier zweimal). Die Zahl der Aufrufe wächst dadurch exponentiell – eine Schleife ist hier viel schneller.

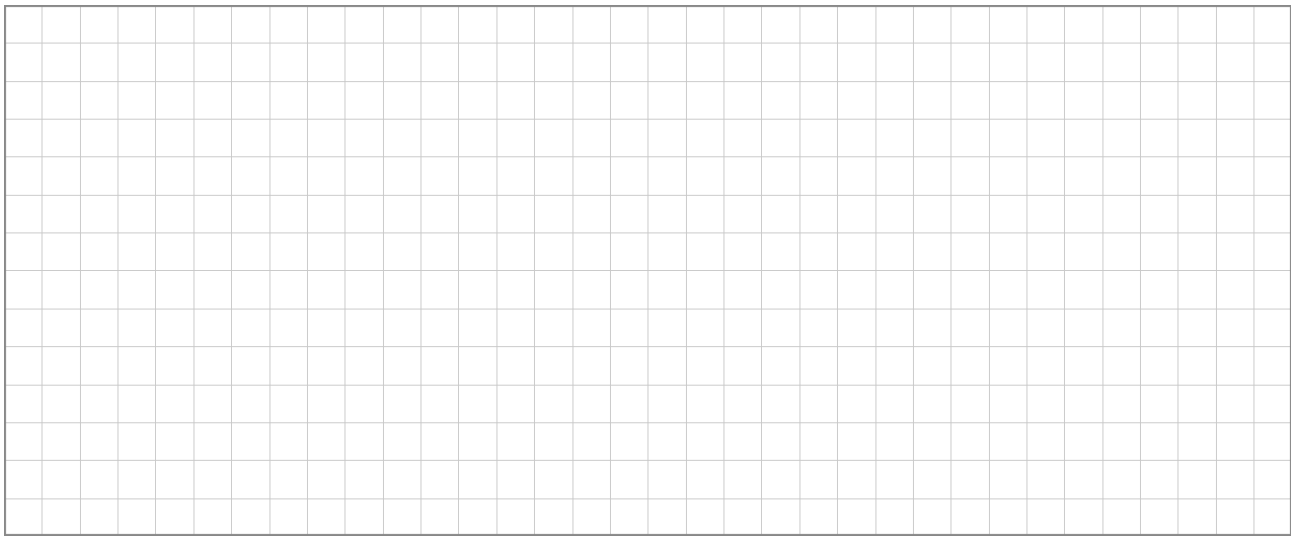
Aufgabe 6: Die Wege durchs Gitterverlies

PAPIER ★★★

Im Gitterverlies startet der Held in der Ecke oben links (0|0) und darf nur nach **rechts** oder nach **unten** gehen. Die Methode `wege(z, s)` liefert, auf wie vielen verschiedenen Wegen er das Feld (z|s) erreichen kann (z, s ≥ 0).

```
public static int wege(int z, int s) {
    if (z == 0 || s == 0) {
        return 1;                    // am Rand: nur ein Weg (immer geradeaus)
    }
    return wege(z - 1, s) + wege(z, s - 1);
}
```

a) Zeichne den Aufrufbaum für `wege(2, 2)` und notiere an jedem Knoten den Rückgabewert. **[3 BE]**



b) Gib die Anzahl aller Aufrufe und die größte Rekursionstiefe für `wege(2, 2)` an. Wie oft wird dabei `wege(1, 1)` aufgerufen? **[2 BE]**

c) Für `wege(3, 3)` gibt es 39 Aufrufe, für `wege(4, 4)` bereits 139. Ein Teammitglied will `wege(10, 10)` berechnen lassen. Beurteile das Vorhaben und schlage eine bessere Lösung vor (Tipp: Welt 3 handelt auch von 2D-Reihungen). **[3 BE]**

Aufgabe 7: Level 3.1 – Die ersten Zauber der Rekursion

IDE ★ – ★★★

Arbeite im Gerüst `AQ_W3_L3_Labyrinth_Geruest.txt` in der Klasse **Rekursion**, Testklasse `L31RekursionTest`. Über jeder Methode stehen Abi-Signatur, Auftrag und der Hinweis **REKURSIV – ohne Schleife**.

Stufe	Methoden (Signatur wie im Abitur)	Tests beginnen mit
★ Basis	fakultaet(n: Ganzzahl): Ganzzahl summeBis(n: Ganzzahl): Ganzzahl potenz(basis: Ganzzahl, exponent: Ganzzahl): Ganzzahl	`basis_`
★★ Erweiterung	anzahlZiffern(n: Ganzzahl): Ganzzahl istPalindrom(s: Zeichenkette): Wahrheitswert summeAb(r: Reihung von Ganzzahl, i: Ganzzahl): Ganzzahl	`erweiterung_`
★★★ Experte	maximumAb(r: Reihung von Ganzzahl, i: Ganzzahl): Ganzzahl dual(n: Ganzzahl): Zeichenkette fibonacci(n: Ganzzahl): Ganzzahl (mit Zähler aufrufe)	`experte_`

a) **Papier first:** Bevor du `summeAb` und `istPalindrom` abtippst, notierst du für je ein eigenes Beispiel Basisfall und Rekursionsschritt in einem Satz.

b) **Regeln:** Zuerst den Basisfall, dann den Rekursionsschritt. Bei `fibonacci` ist `aufrufe++;` die **erste** Anweisung – bei **jedem** Aufruf, auch im Basisfall. Meldet der Testrunner StackOverflowError, prüfst du mit einer Rekursionstabelle, ob der Basisfall erreicht wird.

c) **Vorhersage ohne Gerät:** Das Hauptprogramm gibt `dual(13)` sowie den Wert und die Anzahl der Aufrufe von `fibonacci(6)` aus. Notiere deine Vorhersage, starte dann und vergleiche.

dual(13)	fibonacci(6)	Aufrufe von fibonacci bei fibonacci(6)

Code-Review im Team (das kann kein Test prüfen):

- ☐ Enthält eine Methode mit „REKURSIV – ohne Schleife“ ein `for` oder `while`? Dann ist der Auftrag nicht erfüllt – auch wenn alles grün ist.
- ☐ Ruft sich jede Methode selbst auf, und kommt jeder Selbstaufruf dem Basisfall näher?
- ☐ Wird der Rückgabewert jedes Selbstaufrufs auch **verwendet** (Bug D aus Aufgabe 4)?

Aufgabe 8: Deko-Werkstatt: Fraktale für das Labyrinth

IDE ★ – ★★★

Im Gerüst `AQ_W3_Fraktale_Geruest.txt` zeichnet eine Schildkröte (Turtle) Deko für das Labyrinth: Bäume, Eiszapfen (Koch-Kurve), Schneeflocken, die Portal-Rune (Sierpinski-Dreieck) und den H-Baum. Jede Figur besteht aus kleineren Kopien ihrer selbst. Die Werkstatt gibt nach jedem Bild `Aufrufe: …` und `Striche: …` aus; `Werkstatt.tabelle(deko, bisTiefe)` zählt für mehrere Tiefen auf einmal.

Achtung: Der **Parameter** `tiefe` der Fraktal-Methoden zählt **rückwärts** bis 0 (Basisfall). Er ist nicht die Rekursionstiefe: Ein Aufruf mit dem Parameter tiefe = t hat bei Start mit tiefe = n die Rekursionstiefe n − t.

a) **Deko 1 (Baum, fertig):** Sage zuerst ohne Gerät voraus, wie viele Aufrufe und Striche der Baum für die Tiefen 0 bis 4 erzeugt. Prüfe dann mit `Werkstatt.tabelle("baum", 4)` und stelle eine Formel für die Aufrufe bei Tiefe n auf.

Tiefe n	0	1	2	3	4	Formel für n
Aufrufe (Vorhersage)						
Aufrufe (gemessen)						
Striche (gemessen)						

b) **Deko 2 bis 4:** Programmiere `baum3`, `koch` (mit `schneeflocke`) und `sierpinski` nach den Kommentaren im Gerüst. Prüfe jede Figur mit kleiner Tiefe und langsamem Tempo.

c) **Analyse:** Trage für jede Deko die Anzahl der Selbstaufrufe pro Aufruf (k) und die gemessenen Aufrufe ein. Begründe, um welchen Faktor die Zahl der Aufrufe etwa wächst, wenn die Tiefe um 1 steigt.

Deko	k	Aufrufe Tiefe 2	Tiefe 3	Tiefe 4	Wachstum je Stufe
baum3					
koch					
sierpinski					

d) **Experte:** H-Baum nach dem Kommentar im Gerüst. Wie viele Striche zeichnet er bei Tiefe n?

Für Schnelle: Rätsel aus den Tiefen

- **Türme von Hanoi:** n Scheiben sollen von Stab A nach C, nie eine größere auf eine kleinere. Rekursiver Gedanke: n − 1 Scheiben von A nach B, die größte von A nach C, dann n − 1 Scheiben von B nach C. Implementiere `hanoi(n: Ganzzahl, von: Zeichen, nach: Zeichen, ueber: Zeichen): nichts`, die alle Züge ausgibt, und zähle die Züge für n = 1 bis 10. Wie lange bräuchten Mönche für 64 Scheiben bei einem Zug pro Sekunde?
- **Euklid im Labyrinth:** `ggT(a, b)` ist a für b = 0, sonst `ggT(b, a % b)`. Dokumentiere `ggT(84, 36)` in einer Rekursionstabelle. Warum endet das Verfahren immer?
- **Spiegelwort:** Schreibe `spiegelwort(s: Zeichenkette): Zeichenkette` rekursiv, die s gefolgt von s rückwärts liefert: "abc" → "abccba".

🎯 Im Lab üben

- **Level 3.1 „Spiegelsaal: Basisfall und Rekursionstabelle“** – passt zu Infoteil 1–3 und Aufgaben 0–4
- **Level 3.2 „Hydra: Aufrufbaum und Aufrufe zählen“** – passt zu Infoteil 4–5 und Aufgaben 5–6

Im **Algo Quest Lab** (algoquest.frankjuchim.de) werden die Aufgaben sofort geprüft – **Punkte gibt es dort automatisch**. Anmelden mit deinem Namen und deinem persönlichen Code.

Abi-Check Rekursion – vor jeder Abgabe

- ☐ Basisfall vorhanden und **vor** dem Selbstaufruf geprüft; er deckt alle kleinsten Fälle ab (auch 0, "" und leere Reihung).
- ☐ Jeder Selbstaufruf kommt dem Basisfall näher (n − 1, i + 1, kürzere Teilzeichenkette).
- ☐ Jeder Pfad endet mit einem `return` passenden Typs; der Rückgabewert des Selbstaufrufs wird verwendet.
- ☐ Rekursionstabelle: eine Zeile pro Aufruf in der Reihenfolge ihres Beginns, Erstaufruf = Tiefe 0, Rückgaben von unten nach oben.
- ☐ Anzahl der Aufrufe: Erstaufruf mitzählen.